

1 Теория

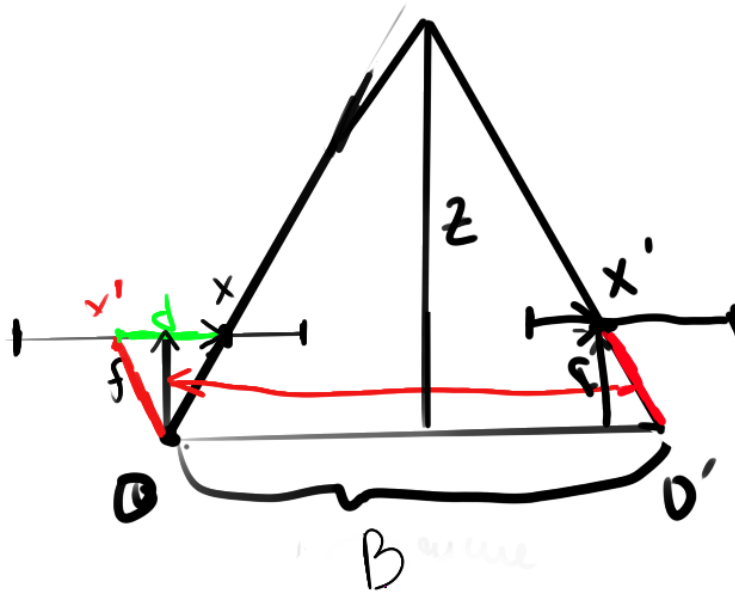


Рис. 1: схема бинокулярного зрения

где f - фокусное расстояние между камерой, O, O' - две наши камеры, Z - реальное расстояние (глубина), d - несоответствие

Используя подобие треугольников можем найти Z по формуле

$$Z = \frac{B \cdot d}{x - x'} = depth$$

2 Практическое применение

2.1 openCV

В нем есть два метода поиска несоответствия:

- stereoBM
- stereoSGBM

Принцип работы обоих разберем на stereoBM, так как принцип почти одинаковый, есть бегущая строчка с окном, в котором считывается наилучшее совпадение на левом и правом кадре, и далее уже по этим пикселям считывает несоответствие и дальше мы получаем карту несоответствия. Использование в коде происходит с помощью команд в кода:

```
stereoBM = cv2.StereoBM_create(numDisparities=128, blockSize=25)
disparityBM = stereoBM.compute(leftImage, rightImage)
```

В первой строчке мы просто создаем объект stereoBM с параметрами, а в следующей строчке мы передаем в этот объект изображение с обеих камер и получаем карту несоответствий. Также у него есть параметры которые можно переделать по надобности.

В коде ниже указано как в дальнейшем имея карту несоответствий перейти к карте глубины

```
depthBM = np.zeros(shape=leftImage.shape).astype('uint8')
depthBM[validPixelsBM] = (f * baseline) /
                        (scale * disparityBM[validPixelsBM])
```

Также есть возможность построить тепловую карту.

2.2 Сравнение методов представленных в OpenCV

- stereoBM

- Плюсы

- * Высокая скорость
 - * Простая настройка
 - * Низкие требования к железу

- Минусы

- * Шумная карта несоответствий
 - * Провалы в однорощднных зонах
 - * Артефакты на границах
 - * Чувствительность к освещению и калибровке

- stereoSGBM

- Плюсы

- * Высокая точность
 - * Сохранение границ объектов
 - * Устойчивость к шуму и слабой текстуре
 - * Предсказуемое поведение при правильной настройке

- Минусы

- * Вычислительно затратный
 - * Больше параметров для точной настройки
 - * Требуется более сильное железо